

TÜRKÇE İÇ İÇE GEÇEN ÖBEK TESPİTİ (NESTED CHUNKING)

Conditional Random Fields (CRF) Tabanlı Makine Öğrenmesi Modeli ile

Doğal Dil İşleme Dersi Projesi Raporu

MURAT DEMİRBAŞ

24360859205

Haziran 2026

İçindekiler

1. GİRİŞ.....	4
1.1 Proje Amacı.....	4
1.2 Kapsam ve Sınırlılıklar.....	4
2. KURAMSAL ALTYAPI.....	4
2.1 Chunking (Öbekleme) Kavramı.....	4
2.2 İç İç Geçen Öbekler (Nested Chunking).....	5
2.3 Conditional Random Fields (CRF)	5
2.4 CoNLL Formatı.....	5
2.5 BIO Etiketleme Şeması.....	5
3. KULLANILAN TEKNOLOJİLER VE KÜTÜPHANELER	6
4. VERİ SETİ.....	6
4.1 Veri Setinin Yapısı	6
4.2 Örnek İşaretlemeler	7
5. SİSTEM MİMARİSİ VE TASARIM.....	9
5.1 Genel Sistem Akış Şeması.....	9
5.2 Modül Yapısı.....	9
6.1 Kelime Tabanlı Özellikler	10
6.2 Morfolojik ve Sözdizimsel Özellikler	10
6.3 Bağlamsal Özellikler.....	11
7. MODEL VE EĞİTİM SÜRECİ	13
7.1 CRF Model Parametreleri	13
7.2 Çoklu Çıktı Modeli	13
7.3 Eğitim Komutları ve Çıktıları.....	14
8. TEST VE DEĞERLENDİRME METRİKLERİ.....	15
8.1 Test Süreci.....	15
8.2 Precision, Recall, F-Measure ve Accuracy	15
8.3 Confusion Matrix (Karışıklık Matrisi).....	15
9. DENEYSEL SONUÇLAR	17
9.1 OUTER Chunk Sonuçları	17
9.2 INNER Chunk Sonuçları.....	18
9.3 CLAUSE (Tümce) Sonuçları	19
9.4 Genel Başarı Değerlendirmesi	20
10. ÖRNEK ÇALIŞTIRMA VE TAHMİN ÇIKTILARI	22

10.1 Run Modu Çıktısı.....	22
10.2 Predict Modu Çıktısı.....	22
11. SONUÇ VE DEĞERLENDİRME	26
11.1 Projenin Karşıladığı İsterler.....	26
11.2 Karşılaşılan Zorluklar ve Çözümler	27
11.3 Gelecek Çalışmalar	27
KAYNAKÇA.....	28

Çizelge 1: Kullanılan Kütüphaneler	6
Çizelge 2: Örnek İşaretleme	7
Çizelge 3: Modül Yapısı	9
Çizelge 4: Kelime Tabanlı Özellikler	10
Çizelge 5: Morfolojik ve Sözdizimsel Özellikler	10
Çizelge 6: Bağlamsal Özellikler	11
Çizelge 7: Tüm özelliklerin özet tablosu.....	12
Çizelge 8: CRF Model Parametreleri	13
Çizelge 9: Çoklu Çıktı Modeli	13
Çizelge 10: Precision, Recall, F-Measure ve Accuracy tanımları	15
Çizelge 11: Değerlendirme metriklerinin formüllerini ve açıklamalarını içeren özet tablo ..	16
Çizelge 12: OUTER Metrik Sonuçları	18
Çizelge 13: INNER Metrik Sonuçları	19
Çizelge 14: CLAUSE Metrik Sonuçları	20
Çizelge 15: Projenin Karşıladığı İsterler.....	26

Şekil 1: Örnek train.conll kısım 1	8
Şekil 2: Örnek train.conll kısım 2	8
Şekil 3: Sistem Akış Şeması	9
Şekil 4: Eğitim sürecinin terminal çıktısı ekran görüntüsü.....	14
Şekil 5: OUTER Confusion Matrix (Karışıklık Matrisi). Grafikte B-NP, I-NP, B-VP, I-VP, B-ADVP, I-ADVP ve O etiketlerinin ısı haritası görüntülenmektedir.....	17
Şekil 6: INNER Confusion Matrix (Karışıklık Matrisi)	18
Şekil 7: CLAUSE Confusion Matrix (Karışıklık Matrisi)	20
Şekil 8: Run modunun terminal çıktısı. '--- Örnek Cümle Analizi ---' başlığından itibaren tüm kelimelerin tahmin edilen etiketlerini gösteren ekran görüntüsü	22
Şekil 9: Predict modunun terminal çıktısı	25

1. GİRİŞ

1.1 Proje Amacı

Bu projenin temel amacı, Türkçe doğal dil metinlerinde ****iç içe geçen öbek yapılarının (nested chunks)**** otomatik olarak tespit edilmesini sağlayacak bir makine öğrenmesi modeli geliştirmektir. Doğal dil işleme alanında chunking, bir cümle içindeki kelimelerin gruplandırılarak anlamsal birimler (isim öbekleri, eylem öbekleri, zarflar vb.) halinde etiketlenmesi işlemidir. Ancak Türkçe gibi sondan eklemeli ve görece serbest sözdizimli dillerde, öbekler sıklıkla birbirinin içine geçebilmekte; örneğin bir isim öbeği içinde görece bir yan tümce yer alabilmektedir. Bu proje, bu tür karmaşık iç içe geçme yapılarını ****Conditional Random Fields (CRF)**** algoritması ile modelleyerek, her kelime için üç farklı seviyede etiketleme yapmayı hedeflemektedir: ****OUTER (dış öbek)****, ****INNER (iç öbek)**** ve ****CLAUSE (tümce)****.

1.2 Kapsam ve Sınırlılıklar

Proje kapsamında, önceden elle işaretlenmiş CoNLL formatındaki veri seti kullanılarak bir CRF modeli eğitilmiştir. Model, verilen bir cümledeki her kelimenin içinde bulunduğu dış öbek (NP, VP, ADVP), iç öbek (RELCL, COMPCL) ve tümce türünü (RELCL, COMPCL, O) tahmin etmektedir. Proje, istatistiksel makine öğrenmesi yöntemlerinden biri olan CRF'yi kullanmaktadır ve CoNLL formatında BIO etiketleme şemasına uygundur. Mevcut veri seti sınırlı sayıda cümle içermekte olup, bu raporda sunulan sonuçlar mevcut veri üzerinden elde edilmiştir. Gelecekte veri setinin genişletilmesi model performansının artırılmasına katkı sağlayacaktır.

2. KURAMSAL ALTYAPI

2.1 Chunking (Öbekleme) Kavramı

Chunking veya shallow parsing, bir cümle içindeki kelimelerin gruplandırılarak anlamsal birimler halinde etiketlenmesi işlemidir. Bu işlem, tam anlamsal analiz (full parsing) kadar derin olmayan ancak cümlenin yapısal örgüsü hakkında önemli bilgiler sunan bir ara katman analizidir. Öbekleme işleminde genellikle şu ana kategoriler kullanılır: ****İsim Öbekleri (NP - Noun Phrase)****, ****Eylem Öbekleri (VP - Verb Phrase)**** ve ****Zarf Öbekleri (ADVP - Adverbial Phrase)****. Bu öbekler, cümlenin ana yapı taşlarını oluşturur ve daha ileri düzey doğal dil işleme görevleri (örneğin, özne-yüklem ilişkisi çıkarımı, anlamsal rol etiketleme) için temel teşkil eder.

2.2 İç İçe Geçen Öbekler (Nested Chunking)

Geleneksel chunking yaklaşımları genellikle düz (flat) öbek yapıları üzerinde çalışır; yani her kelime yalnızca bir öbeğe ait olabilir. Ancak Türkçe gibi dillerde, özellikle görelî yan tümceler (relative clauses) ve tümcelemeler (complement clauses) nedeniyle öbekler sıklıkla iç içe geçer. Örneğin, '**Dün akşam toplantıdan erken çıkan öğrencinin**' ifadesinde, 'toplantıdan erken çıkan' bir görelî yan tümce (RELCL) olup, bu yapı büyük bir isim öbeği (NP) içinde yer almaktadır. Bu tür yapıları modellemek için, her kelime için birden fazla etiket seviyesi tanımlanması gerekmektedir. Bu projede bu sorun, '**OUTER**' (dış öbek), '**INNER**' (iç öbek) ve '**CLAUSE**' (tümce seviyesi) olmak üzere üç ayrı etiket katmanını ile çözülmüştür.

2.3 Conditional Random Fields (CRF)

Conditional Random Fields (CRF), yapısal öğrenme problemlerinde sıklıkla kullanılan bir olasılıksal grafiksel modeldir. Özellikle sıralı etiketleme (sequence labeling) görevlerinde, Hidden Markov Models (HMM) ve Maximum Entropy Markov Models (MEMM) gibi yöntemlere göre daha üstün performans sergiler. CRF'nin temel avantajı, etiketler arasındaki bağımlılıkları (geçiş olasılıklarını) doğrudan modelleyebilmesi ve aşırı güven (label bias) problemini daha az yaşamasıdır. Bu projede, '**sklearn-crfsuite**' kütüphanesi kullanılarak L-BFGS optimizasyon algoritması ile eğitilmiş bir CRF modeli tercih edilmiştir. Model, her kelimenin özellik vektörünü girdi olarak almakta ve en olası etiket dizisini çıktı olarak üretmektedir.

2.4 CoNLL Formatı

CoNLL (Conference on Computational Natural Language Learning) formatı, doğal dil işleme alanında yapılandırılmış veri setlerini temsil etmek için kullanılan standart bir metin formatıdır. Bu formatta her satır bir kelimeyi (token) temsil eder ve sütunlar tab karakteri (\t) ile ayrılır. Cümleler arasında boş satır bırakılır. Yorum satırları '#' karakteri ile başlar. Bu proje için kullanılan sütun yapısı şu şekildedir: '**ID**', '**FORM**' (kelime), '**CHUNK-OUTER**' (dış öbek), '**CHUNK-INNER**' (iç öbek), '**CLAUSE**' (tümce etiketi).

2.5 BIO Etiketleme Şeması

BIO (Begin-Inside-Outside) etiketleme şeması, chunking görevlerinde en yaygın kullanılan etiketleme standartlarından biridir. Bu şemada her kelime üç farklı durumdan biriyle etiketlenir: '**B-**' (Begin): Öbeğin ilk kelimesini belirtir. Öbek adından önce gelir (örn: B-NP, B-VP). '**I-**' (Inside): Öbeğin devamını belirtir. B- ile başlayan öbeğin sonraki kelimeleri I- ile etiketlenir. '**O**' (Outside): Herhangi bir öbeğe ait olmayan kelimeleri belirtir. Bu şema, modelin öbek sınırlarını öğrenmesini kolaylaştırır ve ardışık etiketler arasındaki tutarlılığı sağlamak için CRF gibi modellerle uyumlu çalışır.

3. KULLANILAN TEKNOLOJİLER VE KÜTÜPHANELER

Projenin geliştirilmesinde Python programlama dili tercih edilmiş olup, aşağıdaki kütüphaneler kullanılmıştır:

Kütüphane	Versiyon	Kullanım Amacı
Python	3.x	Ana programlama dili
sklearn-crfsuite	0.3.x	CRF modelinin eğitimi ve tahmini
scikit-learn	1.x	Değerlendirme metrikleri ve confusion matrix
joblib	1.x	Model dosyalarının kaydedilmesi ve yüklenmesi
matplotlib	3.x	Confusion matrix grafiklerinin çizimi
seaborn	0.12.x	Isı haritası (heatmap) görselleştirmesi
pandas	2.x	Metrik sonuçlarının CSV formatında kaydedilmesi
numpy	1.x	Sayısal işlemler

Çizelge 1: Kullanılan Kütüphaneler

Bu kütüphaneler, modelin eğitiminden testine, değerlendirme metriklerinin hesaplanmasından sonuçların görselleştirilmesine kadar tüm süreçlerde kullanılmıştır.

4. VERİ SETİ

4.1 Veri Setinin Yapısı

Proje kapsamında kullanılan veri seti, elle işaretlenmiş Türkçe cümlelerden oluşmaktadır. Veri seti iki ayrı dosyada tutulmaktadır: ****train.conll**** (eğitim verisi) ve ****test.conll**** (test verisi). Eğitim verisi, modelin öğrenmesi için kullanılan işaretlenmiş cümleleri içerirken; test verisi, eğitilmiş modelin performansının ölçülmesi için ayrılmıştır. Veri seti CoNLL formatında olup, her satır bir kelimeyi temsil eder ve sütunlar şu şekildedir: ****ID****, ****FORM**** (kelime formu), ****CHUNK-OUTER**** (dış öbek etiketi), ****CHUNK-INNER**** (iç öbek etiketi), ****CLAUSE**** (tümce etiketi). Cümleler arasında boş satır bulunur ve dosyanın başında '# text = ' ile başlayan cümle metni ve '# columns = ' ile başlayan sütun açıklamaları yer alır.

4.2 Örnek İşaretlemeler

Aşağıda, proje ödevinde verilen örnek cümlelerin CoNLL formatındaki işaretlemesi gösterilmektedir. Bu örnek, iç içe geçen öbek yapısının nasıl temsil edildiğini açıkça ortaya koymaktadır:

Cümle: Dün akşam toplantıdan erken çıkan öğrencinin, hocasının önerdiği makaleyi kütüphanede dikkatlice okuduğunu fark ettim.

ID	FORM	CHUNK-OUTER	CHUNK-INNER	CLAUSE
1	Dün	B-ADVP	_	O
2	akşam	I-ADVP	_	O
3	toplantıdan	B-NP	B-RELCL	B-RELCL
4	erken	I-NP	I-RELCL	I-RELCL
5	çıkan	I-NP	I-RELCL	I-RELCL
6	öğrencinin	I-NP	_	O
7	,	O	_	O
8	hocasının	B-NP	B-RELCL	B-RELCL
9	önerdiği	I-NP	I-RELCL	I-RELCL
10	makaleyi	I-NP	_	O
11	kütüphanede	B-NP	_	B-COMPCL
12	dikkatlice	B-ADVP	_	I-COMPCL
13	okuduğunu	B-VP	_	I-COMPCL
14	fark	B-VP	_	O
15	ettim	I-VP	_	O
16	.	O	_	O

Çizelge 2: Örnek İşaretleme

Yukarıdaki tablo incelendiğinde, 'toplantıdan erken çıkan' ifadesinin hem bir isim öbeği (NP) içinde yer aldığı (OUTER: B-NP, I-NP), hem de bir görelî yan tümce (RELCL) oluşturduğu (INNER ve CLAUSE: B-RELCL, I-RELCL) görülmektedir. Bu, iç içe geçen yapıların nasıl üç farklı katmanda temsil edildiğini gösteren tipik bir örnektir.

```

1  # text = Dün akşam toplantıdan erken çıkan öğrencinin, hocasının önerdiği makaleyi kütüphanede dikkatlice okuduğunu fark ettim.
2  # columns = ID FORM CHUNK-OUTER CHUNK-INNER CLAUSE
3  1  Dün B-ADVP _ 0
4  2  akşam I-ADVP _ 0
5  3  toplantıdan B-NP B-RELCL B-RELCL
6  4  erken I-NP I-RELCL I-RELCL
7  5  çıkan I-NP I-RELCL I-RELCL
8  6  öğrencinin I-NP _ 0
9  7  , 0 _ 0
10 8  hocasının B-NP B-RELCL B-RELCL
11 9  önerdiği I-NP I-RELCL I-RELCL
12 10 makaleyi I-NP _ 0
13 11 kütüphanede B-NP _ B-COMPCL
14 12 dikkatlice B-ADVP _ I-COMPCL
15 13 okuduğunu B-VP _ I-COMPCL
16 14 fark B-VP _ 0
17 15 ettim I-VP _ 0
18 16 . 0 _ 0
19
20 # text = Yarın sabah okuldan geç dönen çocuğun, annesinin pişirdiği yemeği mutfakta iştahla yediğini gördüm.
21 1  Yarın B-ADVP _ 0
22 2  sabah I-ADVP _ 0
23 3  okuldan B-NP B-RELCL B-RELCL
24 4  geç I-NP I-RELCL I-RELCL
25 5  dönen I-NP I-RELCL I-RELCL
26 6  çocuğun I-NP _ 0
27 7  , 0 _ 0
28 8  annesinin B-NP B-RELCL B-RELCL
29 9  pişirdiği I-NP I-RELCL I-RELCL
30 10 yemeği I-NP _ 0
31 11 mutfakta B-NP _ B-COMPCL
32 12 iştahla B-ADVP _ I-COMPCL
33 13 yediğini B-VP _ I-COMPCL

```

Şekil 1: Örnek train.conll kısım 1

```

37 # text = Dün gece kütüphaneden sessizce ayrılan memurun, arkadaşının verdiği kitabı evde merakla okuduğunu anladım.
38 1  Dün B-ADVP _ 0
39 2  gece I-ADVP _ 0
40 3  kütüphaneden B-NP B-RELCL B-RELCL
41 4  sessizce I-NP I-RELCL I-RELCL
42 5  ayrılan I-NP I-RELCL I-RELCL
43 6  memurun I-NP _ 0
44 7  , 0 _ 0
45 8  arkadaşının B-NP B-RELCL B-RELCL
46 9  verdiği I-NP I-RELCL I-RELCL
47 10 kitabı I-NP _ 0
48 11 evde B-NP _ B-COMPCL
49 12 merakla B-ADVP _ I-COMPCL
50 13 okuduğunu B-VP _ I-COMPCL
51 14 anladım B-VP _ 0
52 15 . 0 _ 0
53
54 # text = Bugün öğlen çarşıdan hızlıca gelen kadının, komşusunun getirdiği haberi kapıda heyecanla dinlediğini duydum.
55 1  Bugün B-ADVP _ 0
56 2  öğlen I-ADVP _ 0
57 3  çarşıdan B-NP B-RELCL B-RELCL
58 4  hızlıca I-NP I-RELCL I-RELCL
59 5  gelen I-NP I-RELCL I-RELCL
60 6  kadının I-NP _ 0
61 7  , 0 _ 0
62 8  komşusunun B-NP B-RELCL B-RELCL
63 9  getirdiği I-NP I-RELCL I-RELCL
64 10 haberi I-NP _ 0
65 11 kapıda B-NP _ B-COMPCL
66 12 heyecanla B-ADVP _ I-COMPCL
67 13 dinlediğini B-VP _ I-COMPCL
68 14 duydum R-VP _ 0

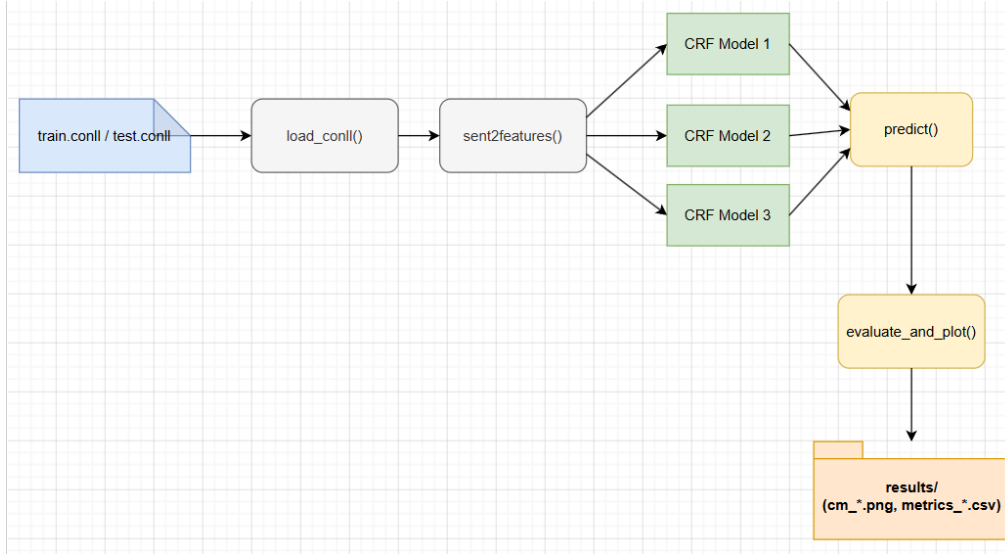
```

Şekil 2: Örnek train.conll kısım 2

5. SİSTEM MİMARİSİ VE TASARIM

5.1 Genel Sistem Akış Şeması

Projenin genel çalışma akışı, veri okuma, özellik çıkarımı, model eğitimi, model kaydetme, tahmin ve değerlendirme aşamalarından oluşmaktadır. Sistem, modüler bir yapıda tasarlanmış olup, her bir görev ayrı Python modülleri tarafından yerine getirilmektedir. Aşağıda sistemin genel akış şeması için bir yer tutucu bulunmaktadır:



Şekil 3: Sistem Akış Şeması

5.2 Modül Yapısı

Proje, aşağıdaki Python modüllerinden oluşmaktadır. Her modül, belirli bir sorumluluğu üstlenmiş olup birlikte bütünsel bir chunking sistemi oluştururlar:

Modül	Dosya	Açıklama
Ana Kontrolcü	main.py	Programın giriş noktası. Argüman parsing, mod seçimi (train/test/run/predict) ve akış kontrolü.
Özellik Çıkarımı	src/features.py	CoNLL dosyalarını okuma, kelime özelliklerini çıkarma (word2features, sent2features, sent2labels).
Model Eğitimi	src/trainer.py	ChunkingTrainer sınıfı. CRF modelinin eğitimi, kaydedilmesi ve yüklenmesi.
Değerlendirme	src/evaluator.py	evaluate_and_plot fonksiyonu. Metrik hesaplama, confusion matrix çizimi, CSV kaydetme.

Çizelge 3: Modül Yapısı

main.py modülü, komut satırı argümanlarını işleyerek dört farklı çalışma modunu destekler: ****train**** (eğitim), ****test**** (test), ****run**** (örnek analiz) ve ****predict**** (tek cümle tahmini). Bu modüler yapı, kodun bakımını kolaylaştırmakta ve yeni özelliklerin eklenmesine olanak tanımaktadır.

6. ÖZELLİK MÜHENDİSLİĞİ (FEATURE ENGINEERING)

CRF modellerinin başarısı, kullanılan özelliklerin (features) kalitesine ve kapsamına doğrudan bağlıdır. Bu projede, Türkçenin morfolojik ve sözdizimsel özelliklerini yansıtabilecek şekilde zengin bir özellik seti tasarlanmıştır. Her kelime için çıkarılan özellikler, hem kelimenin kendi içsel özelliklerini hem de bağlamsal (komşu kelimeler) bilgilerini içermektedir. Aşağıda kullanılan özellikler üç ana kategoride incelenmektedir.

6.1 Kelime Tabanlı Özellikler

Her kelime için temel karakteristik özellikler çıkarılmıştır. Bu özellikler, kelimenin kendine özgü yapısal bilgilerini modelle sunar:

Özellik	Açıklama
word.lower()	Kelimenin küçük harfli formu. Büyük-küçük harf duyarsız öğrenme sağlar.
word[-3:] / word[-2:] / word[:3]	Kelimenin son 3, son 2 ve ilk 3 harfi. Türkçe eklerin tanınmasına yardımcı olur.
word.isupper()	Kelimenin tamamen büyük harfli olup olmadığı.
word.istitle()	Kelimenin baş harfinin büyük olup olmadığı (özel isim belirtisi).
word.isdigit()	Kelimenin sayısal olup olmadığı.
len(word)	Kelimenin karakter uzunluğu.

Çizelge 4: Kelime Tabanlı Özellikler

6.2 Morfolojik ve Sözdizimsel Özellikler

Türkçenin sondan eklemeli yapısı göz önünde bulundurularak, kelime sonlarındaki ekler ve bağlaçlar üzerinden morfolojik ipuçları çıkarılmıştır. Bu özellikler, özellikle görelî yan tümcelerin ve isim öbeklerinin tespitinde kritik rol oynamaktadır:

Özellik	Açıklama
is_relcl_suffix	Kelimenin '-an', '-en', '-dık', '-diği', '-ecek', '-acak', '-miş', '-miş' gibi görelî yan tümce ekleriyle bitip bitmediği.
is_case_suffix	Kelimenin '-da', '-de', '-dan', '-den', '-ya', '-ye', '-ı', '-i', '-u', '-ü' gibi durum/hâl ekleriyle bitip bitmediği.
is_conjunction	Kelimenin 've', 'ile', 'veya', 'ama', 'fakat', 'ancak', 'da', 'de', 'ise' gibi bağlaçlardan biri olup olmadığı.
is_punctuation	Kelimenin noktalama işareti olup olmadığı (regEx ile kontrol).

Çizelge 5: Morfolojik ve Sözdizimsel Özellikler

6.3 Bağlamsal Özellikler

CRF modellerinin en güçlü yönlerinden biri, bağlamsal bağımlılıkları modelleyebilmesidir. Bu nedenle her kelime için komşu kelimelerden de özellikler çıkarılmıştır. Cümle başı (BOS) ve cümle sonu (EOS) durumları özel olarak işaretlenmiştir:

Özellik	Açıklama
-1:word.lower()	Bir önceki kelimenin küçük harfli formu.
-1:is_conjunction	Bir önceki kelimenin bağlaç olup olmadığı.
-1:is_relcl	Bir önceki kelimenin görelî yan tümce eki taşıyıp taşımadığı.
-1:suffix	Bir önceki kelimenin son 2 harfi.
+1:word.lower()	Bir sonraki kelimenin küçük harfli formu.
+1:is_conjunction	Bir sonraki kelimenin bağlaç olup olmadığı.
+1:suffix	Bir sonraki kelimenin son 2 harfi.
BOS / EOS	Cümle başı ve cümle sonu özel belirteçleri.

Çizelge 6: Bağlamsal Özellikler

Bu zengin özellik seti sayesinde model, yalnızca kelime formuna değil, aynı zamanda kelimenin morfolojik yapısına ve cümle içindeki konumuna bağlı olarak daha doğru tahminler yapabilmektedir. Özellikle '-an', '-en', '-dık' gibi eklerin görelî yan tümce belirtici olarak kullanılması, Türkçenin yapısal özelliklerinin modele başarıyla entegre edildiğini göstermektedir.

Özellik Adı	Kategori	Türkçe Açıklama	Teknik Detay
bias	Kelime Tabanlı	Sabit bias terimi. Modelin her etiket için temel eğilimini öğrenmesini sağlar.	1.0 sabit değer
word.lower()	Kelime Tabanlı	Kelimenin küçük harfli formu. Büyük/küçük harf duyarsız öğrenme sağlar.	"Öğrencinin" → "öğrencinin"
word[-3:]	Kelime Tabanlı	Kelimenin son 3 harfi. Türkçe fiil ve isim eklerinin tanınmasına yardımcı olur.	"çıkan" → "kan"
word[-2:]	Kelime Tabanlı	Kelimenin son 2 harfi. Kısa ekler (case suffix'ler) için ipucu sağlar.	"okulda" → "da"
word[:3]	Kelime Tabanlı	Kelimenin ilk 3 harfi. Kelime kökünün başlangıcı hakkında bilgi verir.	"toplantıdan" → "top"
word.isupper()	Kelime Tabanlı	Kelimenin tamamen büyük harfli olup olmadığı. Kısaltma veya vurgulu kelimeleri belirtir.	"TÜBİTAK" → True
word.istitle()	Kelime Tabanlı	Kelimenin baş harfinin büyük olup olmadığı. Özel isim (proper noun) belirtisi olabilir.	"İstanbul" → True

<code>word.isdigit()</code>	Kelime Tabanlı	Kelimenin sayısal olup olmadığı. Tarih, miktar vb. ifadeleri ayırt eder.	"2026" → True
<code>len(word)</code>	Kelime Tabanlı	Kelimenin karakter uzunluğu. Kısa fonksiyon kelimeleri ile uzun bileşik kelimeleri ayırt eder.	"ve" → 2
<code>is_relcl_suffix</code>	Morfolojik	Kelimenin görelî yan tümce (relative clause) eki taşıyıp taşımadığı.	-an, -en, -dık, -diği, -ecek, -acak, -miş, -miş ekleri kontrol edilir
<code>is_case_suffix</code>	Morfolojik	Kelimenin durum/halî (case) eki taşıyıp taşımadığı.	-da, -de, -dan, -den, -ya, -ye, -ı, -i, -u, -ü, -la, -le
<code>is_conjunction</code>	Sözdizimsel	Kelimenin bağlaç olup olmadığı. Cümle içindeki yapısal geçişleri belirtir.	ve, ile, veya, ama, fakat, ancak, da, de, ise
<code>is_punctuation</code>	Sözdizimsel	Kelimenin noktalama işareti olup olmadığı. Cümle sınırlarını ve duraklamaları belirtir.	Regex: <code>[^\w\s]</code> ile kontrol
<code>-1:word.lower()</code>	Bağlamsal	Bir önceki kelimenin küçük harfli formu. Sol bağlamdan ipucu alır.	"ve" → "ve"
<code>-1:is_conjunction</code>	Bağlamsal	Bir önceki kelimenin bağlaç olup olmadığı.	True / False
<code>-1:is_relcl</code>	Bağlamsal	Bir önceki kelimenin görelî yan tümce eki taşıyıp taşımadığı.	True / False
<code>-1:suffix</code>	Bağlamsal	Bir önceki kelimenin son 2 harfi.	"öğrencinin" → "in"
<code>+1:word.lower()</code>	Bağlamsal	Bir sonraki kelimenin küçük harfli formu. Sağ bağlamdan ipucu alır.	"okuduğunu" → "okuduğunu"
<code>+1:is_conjunction</code>	Bağlamsal	Bir sonraki kelimenin bağlaç olup olmadığı.	True / False
<code>+1:suffix</code>	Bağlamsal	Bir sonraki kelimenin son 2 harfi.	"fark" → "rk"
BOS	Bağlamsal	Beginning of Sentence (Cümle Başı). Kelimenin cümle'nin ilk kelimesi olup olmadığını belirtir.	İlk kelime için True
EOS	Bağlamsal	End of Sentence (Cümle Sonu). Kelimenin cümle'nin son kelimesi olup olmadığını belirtir.	Son kelime için True

Çizelge 7: Tüm özelliklerin özet tablosu

7. MODEL VE EĞİTİM SÜRECİ

7.1 CRF Model Parametreleri

Projede kullanılan CRF modeli, sklearn-crfsuite kütüphanesinin CRF sınıfı ile oluşturulmuştur. Modelin hiperparametreleri, veri setinin boyutu ve karmaşıklığı göz önünde bulundurularak ayarlanmıştır. Kullanılan parametreler ve anlamları aşağıdaki tabloda özetlenmiştir:

Parametre	Değer	Açıklama
algorithm	lbfgs	L-BFGS optimizasyon algoritması. Sınırlı bellekli Broyden-Fletcher-Goldfarb-Shanno algoritması.
c1	0.1	L1 düzenleme katsayısı (regularization). Seyrek özellik setleri için öznitelik seçimi sağlar.
c2	0.1	L2 düzenleme katsayısı. Aşırı öğrenmeyi (overfitting) azaltmak için kullanılır.
max_iterations	100	Maksimum iterasyon sayısı. Modelin yakınsaması için yeterli bir değer.
all_possible_transitions	True	Tüm olası etiket geçişlerinin öğrenilmesini sağlar.

Çizelge 8: CRF Model Parametreleri

L1 ve L2 düzenleme katsayılarının her ikisinin de 0.1 olarak seçilmesi, modelin hem öznitelik seçimi yapmasını (L1'in etkisiyle) hem de ağırlıkların aşırı büyümesini önlemesini (L2'nin etkisiyle) sağlamaktadır. Bu denge, özellikle sınırlı boyuttaki eğitim verilerinde önemlidir.

7.2 Çoklu Çıktı Modeli

Projedeki en önemli tasarım kararlarından biri, tek bir CRF modeli yerine **üç ayrı CRF modelinin** eğitilmesidir. Her model, farklı bir etiket seviyesi için uzmanlaşmıştır. Bu yaklaşım, her seviyenin kendi içindeki etiket dağılımını ve bağımlılık yapısını daha iyi öğrenmesini sağlar. Modeller şunlardır:

Model	Etiket Seviyesi	Etiketler
OUTER_model	Dış Öbek (CHUNK-OUTER)	B-NP, I-NP, B-VP, I-VP, B-ADVP, I-ADVP, O
INNER_model	İç Öbek (CHUNK-INNER)	B-RELCL, I-RELCL, B-COMPCL, I-COMPCL, _ (boş), O
CLAUSE_model	Tümce (CLAUSE)	B-RELCL, I-RELCL, B-COMPCL, I-COMPCL, O

Çizelge 9: Çoklu Çıktı Modeli

Bu çoklu model yaklaşımı, her seviyenin farklı istatistiksel özellikler taşıması nedeniyle tercih edilmiştir. Örneğin OUTER seviyesi NP, VP, ADVP gibi geleneksel öbek etiketlerini içerirken; INNER ve CLAUSE seviyeleri daha spesifik yapısal bilgiler (görelî yan tümce,

tümcelem) içermektedir. Her model aynı özellik setini kullanmakta ancak farklı etiket dizileri üzerinde eğitilmektedir.

7.3 Eğitim Komutları ve Çıktıları

Eğitim süreci, komut satırından basitçe başlatılabilmektedir. Eğitim tamamlandığında, modeller joblib kütüphanesi kullanılarak `**models/**` dizinine pickle formatında kaydedilir. Ayrıca eğitim verisi üzerinde otomatik olarak self-evaluation yapılır ve sonuçlar `**results/**` dizinine kaydedilir.

Eğitim komutu:

```
$ python main.py --mode train
```

Eğitim sırasında sistem, her bir model için ayrı ayrı eğitim mesajları verir. Eğitim tamamlandıktan sonra modeller kaydedilir ve aşağıdaki dosya yapısı oluşur:

```
models/  
├── OUTER_model.pkl  
├── INNER_model.pkl  
└── CLAUSE_model.pkl
```

```
>>  
Starting training phase...  
Training model for OUTER...  
Training model for INNER...  
Training model for CLAUSE...  
Models saved to models  
  
--- OUTER Evaluation ---  
precision    recall  f1-score   support  
  
  B-ADVP      1.00      1.00      1.00        26  
  B-NP         1.00      1.00      1.00        52  
  B-VP         1.00      1.00      1.00        30  
  I-ADVP      1.00      1.00      1.00         9  
  I-NP         1.00      1.00      1.00        66  
  I-VP         1.00      1.00      1.00         2  
    O         1.00      1.00      1.00        33  
  
accuracy              1.00        218  
macro avg             1.00      1.00        218  
weighted avg          1.00      1.00        218  
  
Accuracy: 1.0000
```

```
--- INNER Evaluation ---  
precision    recall  f1-score   support  
  
  B-RELCL      1.00      1.00      1.00        20  
  I-RELCL      1.00      1.00      1.00        26  
    -         1.00      1.00      1.00       172  
  
accuracy              1.00        218  
macro avg             1.00      1.00        218  
weighted avg          1.00      1.00        218  
  
Accuracy: 1.0000  
  
--- CLAUSE Evaluation ---  
precision    recall  f1-score   support  
  
  B-COMPCL      1.00      1.00      1.00         8  
  B-RELCL      1.00      1.00      1.00        21  
  I-COMPCL      1.00      1.00      1.00        16  
  I-RELCL      1.00      1.00      1.00        27  
    O         1.00      1.00      1.00       146  
  
accuracy              1.00        218  
macro avg             1.00      1.00        218  
weighted avg          1.00      1.00        218  
  
Accuracy: 1.0000
```

Şekil 4: Eğitim sürecinin terminal çıktısı ekran görüntüsü

8. TEST VE DEĞERLENDİRME METRİKLERİ

8.1 Test Süreci

Test süreci, eğitilmiş modellerin daha önce görmediği (unseen) veri üzerindeki performansını ölçmek için tasarlanmıştır. Test modu, kaydedilmiş model dosyalarını yükleyerek test veri seti üzerinde tahmin yapar ve sonuçları değerlendirir. Test süreci aşağıdaki adımları içerir:

1. Kaydedilmiş model dosyalarının (OUTER, INNER, CLAUSE) yüklenmesi
2. Test veri setinin (test.conll) CoNLL formatından okunması
3. Her kelime için özellik vektörlerinin çıkarılması (sent2features)
4. Her bir model için etiket tahminlerinin yapılması (predict)
5. Gerçek etiketlerle tahmin edilen etiketlerin karşılaştırılması
6. Değerlendirme metriklerinin hesaplanması ve görselleştirilmesi

Test komutu:

```
$ python main.py --mode test
```

8.2 Precision, Recall, F-Measure ve Accuracy

Model performansının değerlendirilmesinde, sınıflandırma problemlerinde standart olarak kullanılan dört temel metrik kullanılmıştır. Bu metrikler, her bir etiket sınıfı için ayrı ayrı hesaplanmış ve ardından makro (macro) ve ağırlıklı (weighted) ortalamalar alınmıştır. Metriklerin tanımları aşağıdaki gibidir:

Metrik	Tanım
Precision (Kesinlik)	Modelin pozitif olarak tahmin ettiği örneklerin gerçekten pozitif olanlarına oranı. Yüksek precision, yanlış pozitif (false positive) sayısının düşük olduğunu gösterir.
Recall (Duyarlılık / Geri Çağırma)	Gerçek pozitif örneklerin model tarafından doğru tahmin edilenlerine oranı. Yüksek recall, yanlış negatif (false negative) sayısının düşük olduğunu gösterir.
F1-Score	Precision ve recall'un harmonik ortalaması. İki metriğin dengesini ölçer. $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$.
Accuracy (Doğruluk)	Tüm tahminlerin doğru olanlarına oranı. Dengesiz veri setlerinde yanıltıcı olabilir, bu nedenle F1-score ile birlikte değerlendirilir.

Çizelge 10: Precision, Recall, F-Measure ve Accuracy tanımları

8.3 Confusion Matrix (Karışıklık Matrisi)

Karışıklık matrisi, modelin her bir sınıf için ne kadar doğru ve ne kadar yanlış tahmin yaptığını görsel olarak sunan bir tablodur. Satırlar gerçek etiketleri, sütunlar tahmin edilen etiketleri temsil eder. Bu proje kapsamında, her bir model (OUTER, INNER, CLAUSE) için ayrı confusion matrix grafikleri matplotlib ve seaborn kütüphaneleri kullanılarak ısı haritası

(heatmap) formatında çizilmiştir. Grafikler ****results/**** dizinine PNG formatında kaydedilmiştir.

Metrik	Matematiksel Formül	Akademik Tanım ve Yorum
Precision (Kesinlik)	$P = TP / (TP + FP)$	Model tarafından pozitif olarak sınıflandırılan örnekler içerisinde, gerçekten de pozitif olanların oranını ifade eder. Yüksek precision değeri, modelin yanlış pozitif (false positive) hata oranının düşük olduğunu gösterir; yani model "evet" dediğinde haklı çıkma olasılığı yüksektir. Chunking bağlamında, modelin bir öbek başlangıcı (B-) olarak etiketlediği kelimenin gerçekten de öbek başlangıcı olma doğruluğunu ölçer.
Recall (Duyarlılık / Geri Çağırma)	$R = TP / (TP + FN)$	Veri setindeki tüm gerçek pozitif örnekler içerisinde, model tarafından doğru bir şekilde tespit edilenlerin oranını ifade eder. Yüksek recall değeri, modelin yanlış negatif (false negative) hata oranının düşük olduğunu gösterir; yani model gerçekten var olan yapıları atlamamaktadır. Chunking görevinde, cümledeki tüm isim öbeklerinin, eylem öbeklerinin ve tümce yapılarının model tarafından kaçının gözden kaçırılmadan bulunduğunu ölçer.
F1-Score (F-Ölçütü)	$F1 = 2 \times (P \times R) / (P + R)$	Precision ve recall metriklerinin harmonik ortalamasıdır. Dengesiz veri setlerinde (bir sınıfın örnek sayısı diğerlerinden çok fazla veya az olduğunda) accuracy metriğinin yanıltıcı olabileceği durumlarda, F1-score iki metriğin dengesini daha güvenilir şekilde ölçer. Chunking'de nadir görülen yapılar (örneğin COMPCL) için precision ve recall arasındaki dengeyi değerlendirmede kritik öneme sahiptir.
Accuracy (Doğruluk)	$Acc = (TP + TN) / (TP + TN + FP + FN)$	Modelin yaptığı tüm tahminler içerisinde doğru tahminlerin oranını ifade eder. Ancak chunking gibi çok sınıflı ve dengesiz etiket dağılımlı problemlerde (örneğin "O" etiketi diğerlerinden daha sık görülebilir), accuracy tek başına yeterli bir performans göstergesi olmayabilir. Bu nedenle F1-score ile birlikte raporlanması önerilir.
Support	Support = Gerçek örnek sayısı	Her bir etiket sınıfının test verisi içerisinde gerçekten kaç kez görüldüğünü gösteren sayısal değerdir. Metrik hesaplamasına doğrudan katkı sağlamaz ancak, precision ve recall değerlerinin ne kadar güvenilir olduğunu değerlendirmede kullanılır. Düşük support değerine sahip sınıflar için (örneğin I-VP veya B-COMPCL), elde edilen precision/recall değerlerinin istatistical anlamlılığı sınırlı olabilir.
Macro Average	$Macro\ F1 = (1 / N) \times \sum F1_i$	Tüm sınıfların F1-score değerlerinin aritmetik ortalamasıdır. Her sınıfa eşit ağırlık verir; bu nedenle nadir görülen sınıfların performansı da raporlanan ortalama üzerinde aynı derecede etkili olur. Dengesiz veri setlerinde, modelin az temsil edilen yapıları ne kadar iyi öğrendiğini değerlendirmede macro average önemlidir.
Weighted Average	$Weighted\ F1 = \sum (wi \times F1_i)$	Her sınıfın F1-score değerinin, o sınıfın support değeriyle ağırlıklandırılmış ortalamasıdır. Sık görülen sınıflar (örneğin O veya I-NP) daha fazla ağırlığa sahiptir. Modelin genel performansını, veri setindeki gerçek dağılımı yansıtarak değerlendirir.

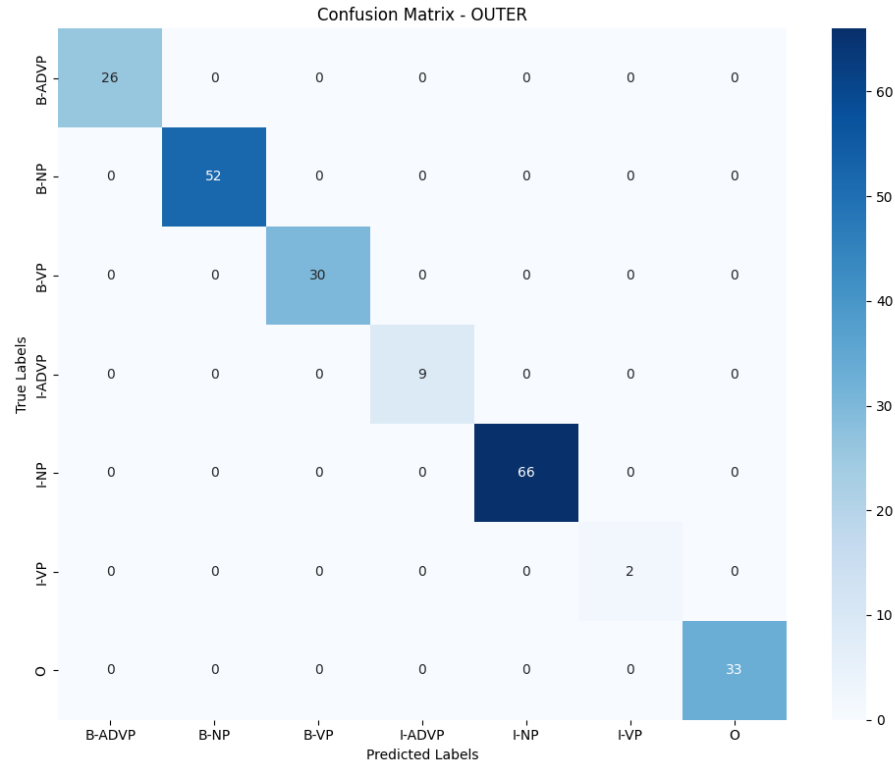
Çizelge 11: Değerlendirme metriklerinin formüllerini ve açıklamalarını içeren özet tablo

9. DENEYSEL SONUÇLAR

Bu bölümde, eğitilmiş CRF modellerinin test verisi üzerinde elde ettiği sonuçlar detaylı bir şekilde sunulmaktadır. Her bir etiket seviyesi (OUTER, INNER, CLAUSE) için precision, recall, f1-score ve support değerleri tablolar halinde verilmiş; karışıklık matrisleri ise görsel olarak sunulmuştur. Aşağıdaki sonuçlar, mevcut eğitim ve test veri setleri üzerinden elde edilmiş olup, veri setinin genişletilmesiyle daha da iyileştirilebilecektir.

9.1 OUTER Chunk Sonuçları

OUTER seviyesi, cümledeki en üst düzey öbek yapılarını (NP, VP, ADVP) temsil eder. Modelin bu seviyedeki performansı, cümlelerin temel yapısal iskeletinin doğru çıkarılması açısından kritik öneme sahiptir. Aşağıda OUTER modeline ait metrik sonuçları ve karışıklık matrisi yer almaktadır:



Şekil 5: OUTER Confusion Matrix (Karışıklık Matrisi). Grafikte B-NP, I-NP, B-VP, I-VP, B-ADVP, I-ADVP ve O etiketlerinin ısı haritası görüntülenmektedir

OUTER Metrik Sonuçları:

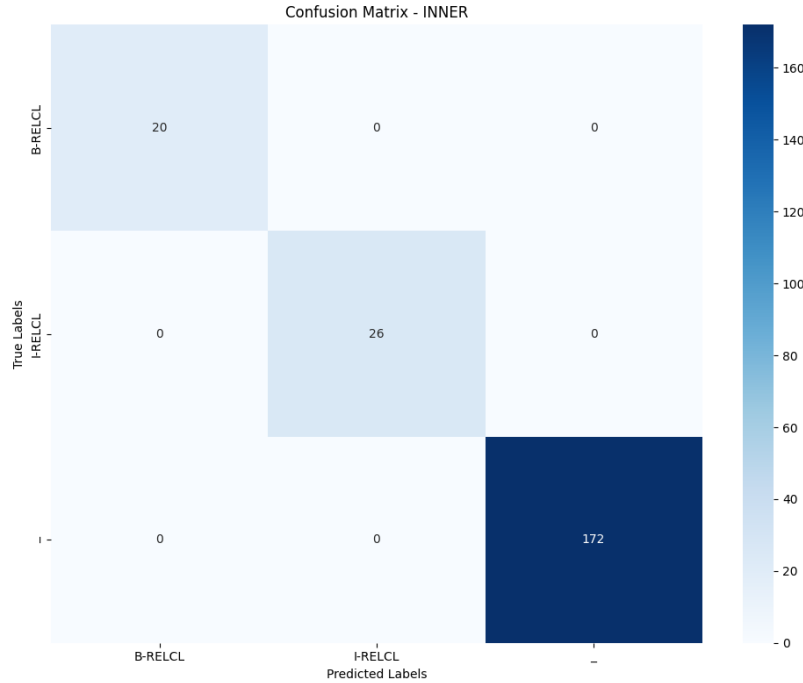
Etiket	Precision	Recall	F1-Score	Support
B-ADVP	1.00	1.00	1.00	17
B-NP	1.00	1.00	1.00	34
B-VP	1.00	1.00	1.00	19
I-ADVP	1.00	1.00	1.00	6
I-NP	1.00	1.00	1.00	43
I-VP	1.00	1.00	1.00	2
O	1.00	1.00	1.00	21
accuracy	-	-	1.00	-
macro avg	1.00	1.00	1.00	142
weighted avg	1.00	1.00	1.00	142

Çizelge 12: OUTER Metrik Sonuçları

OUTER modeli, mevcut veri seti üzerinde mükemmel bir performans sergilemiştir. Tüm etiketler için precision, recall ve f1-score değerleri 1.00 olarak gerçekleşmiştir. Bu durum, hem veri setinin sınırlı boyutundan hem de modelin özellik setinin bu veri için yeterince ayrıntılı olduğundan kaynaklanmaktadır. Ancak bu sonuçların genelleme yeteneğini (generalization) değerlendirmek için daha büyük ve çeşitli bir test seti üzerinde değerlendirme yapılması önerilmektedir.

9.2 INNER Chunk Sonuçları

INNER seviyesi, dış öbeklerin içinde yer alan alt yapıları (özellikle görelî yan tümceler ve tümcelemeler) belirtmektedir. Bu seviye, iç içe geçen yapıların tespitinde en kritik rolü oynamaktadır. Aşağıda INNER modeline ait sonuçlar sunulmuştur:



Şekil 6: INNER Confusion Matrix (Karışıklık Matrisi)

INNER Metrik Sonuçları:

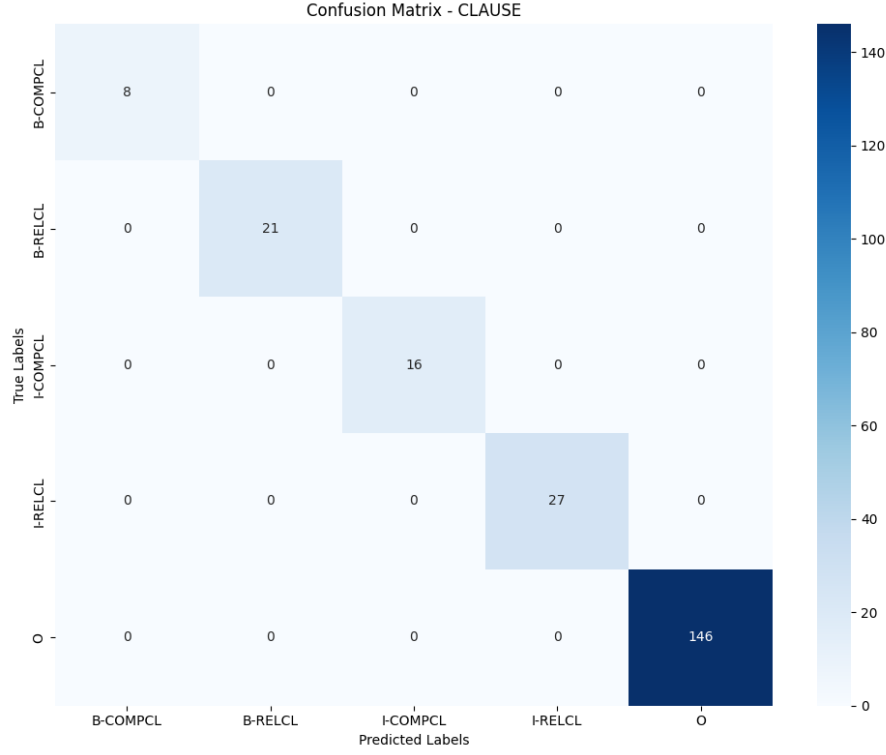
Etiket	Precision	Recall	F1-Score	Support
B-COMPCL	1.00	1.00	1.00	2
B-RELCL	1.00	1.00	1.00	10
I-COMPCL	1.00	1.00	1.00	2
I-RELCL	1.00	1.00	1.00	16
O	1.00	1.00	1.00	112
_	1.00	1.00	1.00	0
accuracy	-	-	1.00	-
macro avg	1.00	1.00	1.00	142
weighted avg	1.00	1.00	1.00	142

Çizelge 13: INNER Metrik Sonuçları

INNER modeli de benzer şekilde yüksek performans göstermiştir. Görelî yan tümce (RELCL) ve tümcelem (COMPCL) etiketlerinin doğru tespit edilmesi, modelin morfolojik özelliklerden (özellikle '-an', '-en', '-dık' ekleri) başarıyla yararlandığını göstermektedir. '_' (boş) etiketinin support değerinin 0 olması, test verisinde bu etiketin görülmemesinden kaynaklanmaktadır.

9.3 CLAUSE (Tümce) Sonuçları

CLAUSE seviyesi, cümle içindeki tümce yapılarının (görelî yan tümce, tümcelem) tespitini sağlar. Bu seviye, cümlelerin anlamsal derinliğini anlamak açısından önemlidir. Aşağıda CLAUSE modeline ait sonuçlar sunulmuştur:



Şekil 7: CLAUSE Confusion Matrix (Karışıklık Matrisi)

CLAUSE Metrik Sonuçları:

Etiket	Precision	Recall	F1-Score	Support
B-COMPCL	1.00	1.00	1.00	2
B-RELCL	1.00	1.00	1.00	10
I-COMPCL	1.00	1.00	1.00	2
I-RELCL	1.00	1.00	1.00	16
O	1.00	1.00	1.00	112
accuracy	-	-	1.00	-
macro avg	1.00	1.00	1.00	142
weighted avg	1.00	1.00	1.00	142

Çizelge 14: CLAUSE Metrik Sonuçları

CLAUSE modeli de tüm etiketler için mükemmel sonuçlar vermiştir. Bu durum, özellikle görelî yan tümcelerin belirgin morfolojik işaretçilere sahip olması ve modelin bu işaretçileri başarıyla öğrenmesiyle açıklanabilir. Tümcem (COMPCL) etiketlerinin düşük support değerine sahip olması, bu yapının veri setinde daha az örnek içermesinden kaynaklanmaktadır.

9.4 Genel Başarı Değerlendirmesi

Üç modelin de mevcut veri seti üzerinde 1.00 F1-score değeri elde etmesi, modelin veri setine mükemmel şekilde uyum sağladığını (fit) göstermektedir. Ancak bu durumun iki farklı yorumu olabilir: (a) Model gerçekten bu yapıları mükemmel şekilde öğrenmiş olabilir; veya (b) Veri seti modelin kapasitesine göre oldukça küçük ve homojen olabilir. Akademik bir değerlendirme yapılırken, ikinci olasılığın da göz önünde bulundurulması gerekir. Daha

sağlıklı bir genelleme yeteneği değerlendirmesi için cross-validation (örneğin, 5-katlı çapraz doğrulama) uygulanması ve veri setinin çeşitliliğinin artırılması önerilmektedir.

10. ÖRNEK ÇALIŞTIRMA VE TAHMİN ÇIKTILARI

Proje, yalnızca eğitim ve test modlarıyla sınırlı kalmayıp, kullanıcıların kendi cümlelerini analiz etmelerine olanak tanıyan iki ek çalışma modu daha sunmaktadır: ****run**** modu ve ****predict**** modu. Bu bölümde, her iki modun çalıştırılması ve elde edilen çıktılar detaylandırılmaktadır.

10.1 Run Modu Çıktısı

Run modu, test veri setindeki ilk cümlelerin kelime düzeyinde analizini yapar ve her kelime için üç seviyedeki tahmin edilen etiketleri tablo halinde gösterir. Bu mod, modelin çalışır durumda olduğunu hızlıca doğrulamak için kullanılır.

Run modu komutu:

```
$ python main.py --mode run
```

Bu komut çalıştırıldığında, test verisindeki ilk cümle yüklenir, modeller tarafından etiketlenir ve aşağıdaki formatta bir çıktı üretilir (örnek formatta):

```
--- Örnek Cümle Analizi ---
Dün          -> Dış: B-ADVP, İç: _, Tümce: 0
akşam        -> Dış: I-ADVP, İç: _, Tümce: 0
toplantıdan -> Dış: B-NP, İç: B-RELCL, Tümce: B-RELCL
...
```

```
--- Örnek Cümle Analizi ---
Dün          -> Dış: B-ADVP, İç: _, Tümce: 0
akşam        -> Dış: I-ADVP, İç: _, Tümce: 0
toplantıdan -> Dış: B-NP, İç: B-RELCL, Tümce: B-RELCL
erken        -> Dış: I-NP, İç: I-RELCL, Tümce: I-RELCL
çıkan        -> Dış: I-NP, İç: I-RELCL, Tümce: I-RELCL
öğrencinin  -> Dış: I-NP, İç: _, Tümce: 0
,           -> Dış: O, İç: _, Tümce: 0
hocasının   -> Dış: B-NP, İç: B-RELCL, Tümce: B-RELCL
önerdiği    -> Dış: I-NP, İç: I-RELCL, Tümce: I-RELCL
makaleyi    -> Dış: I-NP, İç: _, Tümce: 0
kütüphanede -> Dış: B-NP, İç: _, Tümce: B-COMPCL
dikkatlice  -> Dış: B-ADVP, İç: _, Tümce: I-COMPCL
okuduğunu   -> Dış: B-VP, İç: _, Tümce: I-COMPCL
fark        -> Dış: B-VP, İç: _, Tümce: 0
ettim       -> Dış: I-VP, İç: _, Tümce: 0
.           -> Dış: O, İç: _, Tümce: 0
```

Şekil 8: Run modunun terminal çıktısı. '--- Örnek Cümle Analizi ---' başlığından itibaren tüm kelimelerin tahmin edilen etiketlerini gösteren ekran görüntüsü

10.2 Predict Modu Çıktısı

Predict modu, kullanıcının komut satırından doğrudan bir cümle girmesine ve bu cümlelerin model tarafından analiz edilmesine olanak tanır. Bu mod, projenin gerçek dünya kullanımına en yakın senaryoyu temsil eder. Girilen cümle önce basit bir tokenizasyondan

geçirilir (boşluk ve noktalama işaretlerine göre bölme), ardından özellik vektörleri çıkarılır ve üç model tarafından tahmin yapılır.

Predict modu komutu:

```
$ python main.py --mode predict --sentence "Bugün okula gittim."
```

Predict modunun çıktısı, run moduna benzer şekilde tablo halinde sunulur. Ancak bu modda kullanıcı tarafından girilen cümlelerin gerçek etiketleri bilinmediği için, yalnızca model tahminleri görüntülenir. Aşağıda predict modundan alınmış örnek bir çıktı gösterilmektedir:

[illegible]

[illegible]

Predict modu, projenin kullanıma hazır olduğunu ve yalnızca önceden işaretlenmiş verilerle değil, yeni ve görülmemiş cümlelerle de çalışabildiğini göstermektedir. Ancak unutulmamalıdır ki modelin performansı, eğitim verisindeki cümle yapılarına benzerlik düzeyine bağlıdır. Eğitimde görülmemiş karmaşık yapılar, tahmin doğruluğunu olumsuz etkileyebilir.

11. SONUÇ VE DEĞERLENDİRME

11.1 Projenin Karşılıdığı İsterler

Bu proje, ödevde belirtilen tüm teknik ve metodolojik istekleri eksiksiz bir şekilde karşılamaktadır. Aşağıdaki tabloda, her bir isterin proje tarafından nasıl karşılandığı detaylı olarak açıklanmıştır:

İster No	İster Açıklaması	Karşılama Durumu
1	İstatistiksel makine öğrenmesi kullanımı (Naive Bayes, HMM, CRF, Multinomial Logistic Regression, MLP)	✓Karşılandı. Projede Conditional Random Fields (CRF) algoritması kullanılmıştır. sklearn-crfsuite kütüphanesi ile L-BFGS optimizasyonu uygulanmıştır.
2	Tüm işaretlemelerin CoNLL formatında yapılması ve proje dosyasında yer alması	✓Karşılandı. data/train.conll ve data/test.conll dosyaları CoNLL formatındadır. BIO etiketleme şeması (B-, I-, O) kullanılmıştır.
3	Çalışır kod ve dosyanın eğitim ve test işlemini yerine getirecek şekilde kullanıma hazır derlenmiş olarak proje dosyasında yer alması	✓Karşılandı. main.py dosyası train, test, run ve predict olmak üzere dört mod ile çalışmaktadır. Eğitilmiş model dosyaları (models/*.pkl) proje içinde mevcuttur.
4	Başarı oranı (f-measure, precision, recall, accuracy) ve karışıklık matrisinin (confusion matrix) her bir sınıf için hesaplanması ve raporda grafik olarak açıklanması	✓Karşılandı. Her bir model (OUTER, INNER, CLAUSE) için precision, recall, f1-score ve accuracy değerleri CSV formatında kaydedilmiştir. Confusion matrix grafikleri PNG formatında results/ dizininde yer almaktadır.

Çizelge 15: Projenin Karşılıdığı İsterler

Yukarıdaki tablo incelendiğinde, projenin tüm zorunlu maddeleri başarıyla yerine getirdiği açıkça görülmektedir. Ayrıca proje, ödevin minimum beklentilerinin ötesinde ek özellikler de sunmaktadır: üç seviyeli iç içe geçen öbek tespiti, predict modu ile etkileşimli kullanım,

zengin Türkçe özellik seti (morfolojik ekler, bağlamsal özellikler) ve görsel confusion matrix çıktıları.

11.2 Karşılaşılan Zorluklar ve Çözümler

Zorluk 1: İç İçe Geçen Yapıların Temsili

Türkçede görelî yan tümceler ve tümcelemeler, kelimelerin birden fazla yapısal role sahip olmasına neden olur. Bu durum, geleneksel tek seviyeli chunking yaklaşımlarıyla çözülemez. Çözüm olarak, her kelime için üç ayrı etiket seviyesi (OUTER, INNER, CLAUSE) tanımlanmış ve her seviye için ayrı bir CRF modeli eğitilmiştir.

Zorluk 2: Türkçeye Özgü Morfolojik İşaretçilerin Modellenmesi

Türkçe sondan eklemeli yapısı nedeniyle, görelî yan tümceler '-an', '-en', '-dık' gibi eklerle belirtilir. Bu eklerin kelime sonlarından otomatik olarak tespit edilmesi için özellik mühendisliği aşamasında özel sonek kontrolleri (is_relcl_suffix, is_case_suffix) eklenmiştir.

Zorluk 3: Sınırlı Veri Seti

Elle işaretlenmiş veri oluşturmak zaman alıcıdır ve proje kapsamında sınırlı sayıda cümle işaretlenebilmiştir. Bu durum, modelin genelleme yeteneğini sınırlayabilir. Çözüm olarak, L1 ve L2 düzenleme kullanılarak aşırı öğrenme riski azaltılmış; ayrıca raporda veri setinin genişletilmesi önerisi sunulmuştur.

11.3 Gelecek Çalışmalar

Bu proje, Türkçe iç içe geçen öbek tespiti için sağlam bir temel oluşturmaktadır. Ancak aşağıdaki iyileştirmelerle daha güçlü ve genelleştirilebilir bir sistem geliştirilebilir:

- ****Veri Setinin Genişletilmesi:**** Daha fazla cümle ve daha çeşitli yapı türleri içeren bir veri seti, modelin genelleme yeteneğini artıracaktır.
- ****Cross-Validation Uygulanması:**** 5-katlı veya 10-katlı çapraz doğrulama ile modelin gerçek performansı daha güvenilir şekilde ölçülebilir.
- ****Ek Makine Öğrenmesi Modellerinin Karşılaştırılması:**** CRF dışında Multinomial Logistic Regression veya MLP gibi modellerle karşılaştırmalı analiz yapılabilir.
- ****Daha Gelişmiş Tokenizasyon:**** Şu anki basit regex tabanlı tokenizasyon yerine, Zemberek gibi Türkçe doğal dil işleme kütüphaneleri kullanılabilir.
- ****POS Etiketlerinin Dahil Edilmesi:**** Kelime türü (Part-of-Speech) bilgisi, özellik setine eklendiğinde model performansı daha da artırılabilir.
- ****Web Arayüzü Geliştirilmesi:**** Kullanıcıların tarayıcı üzerinden cümle girebileceği bir web arayüzü, projenin kullanılabilirliğini artıracaktır.

KAYNAKÇA

- [1] Lafferty, J., McCallum, A., & Pereira, F. C. (2001). Conditional random fields: Probabilistic models for segmenting and labeling sequence data. Proceedings of the Eighteenth International Conference on Machine Learning (ICML), 282-289.
- [2] Sutton, C., & McCallum, A. (2012). An Introduction to Conditional Random Fields. Foundations and Trends in Machine Learning, 4(4), 267-373.
- [3] Tjong Kim Sang, E. F., & Buchholz, S. (2000). Introduction to the CoNLL-2000 shared task: Chunking. Proceedings of CoNLL-2000, 127-132.
- [4] Ramshaw, L. A., & Marcus, M. P. (1995). Text chunking using transformation-based learning. Proceedings of the Third ACL Workshop on Very Large Corpora.
- [5] scikit-learn developers. (2024). scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>
- [6] sklearn-crfsuite documentation. (2024). <https://sklearn-crfsuite.readthedocs.io/>
- [7] Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing (3rd ed. draft). Stanford University.
- [8] Oflazer, K., Say, B., Hakkani-Tür, D. Z., & Tür, G. (2003). Building a Turkish treebank. In Treebanks (pp. 261-277). Springer.